

Audit tecnico del tracking della pallina in RF Coach

Codice analizzato: commit 616406d (6 ottobre 2026), letto sul PC. Il codice non è stato modificato. Le verifiche sui dati usano i risultati della catena attuale sui 7 video di prova (Nicola, Djokovic, swing_vision, alcaraz, test_tennis_1, Giorgio, video_alcaraz_palline_sparse), ottenuti con gli stessi file del PC.

0. In breve

- **Verdetto: PARZIALMENTE IMPLEMENTATO.**
- Oggi il sistema è soprattutto **detection**. TrackNet dà **un solo punto per fotogramma**: la macchia più grande della heatmap. Su quei punti lavorano filtri e piccoli tracker locali, ognuno nel suo file e con le sue regole.
- Un vero **tracking** (la pallina del fotogramma prima, una previsione, una distanza massima) c'è solo dentro la finestra di ogni colpo, in `palla_locale.segui`, e solo per pochi frame. A ogni colpo riparte da zero.
- Le palline ferme non si evitano **scegliendo** la pallina giusta: si **cancellano** i fotogrammi in cui TrackNet le indica. In quei fotogrammi anche la pallina vera è persa, perché il secondo candidato viene buttato già in `tracknet3/predict.py`.

1. La pipeline: file e funzioni

Fase	File e funzioni	Cosa fa con la pallina	Punti su cui lavora
TrackNet	<code>ball_tracknet.compute_ball_trajectory → tracknet3/predict.py (predict), tracknet3/infer_utils.py (predict_location, get_ensemble_weight, generate_inpaint_mask)</code>	Ingresso: 8 fotogrammi più l'immagine mediana dello sfondo (<code>bg_mode = "concat"</code>). In modalità "weight" la heatmap di ogni frame è la media pesata di 8 finestre sovrapposte. Poi soglia 0,5 e contorni: si tiene solo il contorno di area maggiore , cioè un punto. InpaintNet (16 frame) riempie i buchi con punti di fonte "stimata".	Video intero
Pesi	<code>tracknet3/ckpts/TrackNet_best.pt (2023), InpaintNet_best.pt</code>	Pesi pubblici del badminton (letti dal checkpoint: <code>seq_len 8, bg_mode concat</code>). Il fine-tuning sul tennis (<code>tracknet_finetune/</code>) non è integrato. TrackNetV4 non c'è nel repository : si usa TrackNetV3.	
<code>analyze.py</code>	<code>ball_positions.get(frame_number - 1), ball_speeds, tracking_row, print_detection_summary</code>	Scrive nel tracking CSV <code>pallina_x, pallina_y, pallina_conf, pallina_fonte, pallina_velocita_px_s</code> . <code>BALL_JUMP_WARN_PX = 500</code> stampa solo un avviso e non corregge niente.	Tutti i punti
Colpi dalla posa	<code>classificazione/classifica_tracking.py (colpi.csv frame per frame), velocita_uscita.finestre_colpi</code>	Crea le finestre dei colpi. La pallina qui non entra.	—
Pipeline parallela	<code>classificazione/rileva_impatti.py (trova_impatti), metriche_colpi.py (cella di Colab della classificazione)</code>	Trova gli impatti dalle inversioni della <code>y</code> della pallina su tutto il video, interpolando i buchi (<code>np.interp</code>).	Punti TrackNet grezzi , senza filtro delle palline ferme
Lettura	<code>velocita_uscita.leggi_tracking</code>	<code>vista = fonte == "tracknet"</code> : i punti "stimata" di InpaintNet non si usano mai in <code>velocita/</code> .	Solo i punti di TrackNet
Filtro palline ferme	<code>palla_locale.tracknet_fermi (+ macchie_gialle), chiamato da velocita_uscita.analizza; salva_palline_ferme e leggi_palline_ferme</code>	Cancella i frame in cui il punto di TrackNet è una pallina ferma.	Finestre dei colpi ± 10 frame, più 1,6 s dopo ogni riga

Fase	File e funzioni	Cosa fa con la pallina	Punti su cui lavora
Giallo fermo	palla_locale.giallo_fermo	Maschera del giallo presente nella mediana di 15 fotogrammi del tratto. Vale solo per il rilevatore di colore.	Rilevatore di colore
Tracking locale	palla_locale.segui (+ candidati)	Seme, previsione a velocità costante, prima il colore e poi TrackNet, cancello, al massimo 8 frame di buco.	Punti filtrati + colore
Contatto	velocita_uscita.trova_contatti	Cambio di velocità $\geq 2,5$ altezze/s, pallina entro 0,6 altezze da un polso, dopo il contatto $\geq 2,4$ altezze/s, contatti ad almeno 20 frame l'uno dall'altro.	La traccia di segui
Velocità e direzione	velocita_uscita.misura → rf_ball_exit_speed.exit_speed (refine_impact_visual); controlla_risultato	Fit fisico 3D sui punti da $fc-8$ a $fc+1/3$ s. Se l'errore supera 6 px (a 1080p) la misura è scartata.	La traccia di segui
Solo direzione	velocita_uscita.solo_direzione → segui(esteso=True), seme_lancio	Accetta la pallina mossa dal colpo e la riaggancia a TrackNet dopo un buco.	Punti filtrati
Contatto nascosto	direzione_nascosta.aggiorna: segui (rifatto), punti_dopo (RANSAC), si_allontana, direzione. Dal 6/10 anche prova_in_uscita: tracce, pallina_in_uscita, direzione_in_uscita	Direzione dai punti che ricompaiono dopo il tratto coperto.	Punti filtrati (dal file)
Rimbalzo	velocita_rimbalzo.aggiorna: trova_rimbalzo, inizio_traiettoria, stima (passi 1-3); estendi_colore, macchie_lontane, stima_colore (4); cerca_rimbalzo_coperto, stima_ricostruita (5); rimbalzo_dopo_colpo, stima_da_contatto_esteso (6); aggiungi_media (7)	Rimbalzo, stima dal rimbalzo, punto per la mappa.	Punti filtrati + colore
Video	disegna_velocita.disegna_tracking	Solo disegno: anelli gialli (TrackNet), rossi (punti usati), grigi (palline ferme tolte).	—

Funzioni che riattaccano o ritrovano la pallina:

- **segui:** dopo un buco, cancello attorno alla previsione; in modalità estesa, RIAGGANCI0_REL.
- **punti_dopo:** la pallina che ricompare.
- **tracce e pallina_in_uscita.**
- **estendi_colore.**
- **cerca_rimbalzo_coperto.**
- **inizio_traiettoria:** risale la traiettoria all'indietro dal rimbalzo.

La racchetta (riquadro YOLO nel tracking CSV) non è usata da nessun file di velocita/.

2. A, B o C?

Risposta: C, ma molto spostata verso A.

- **A, al livello di TrackNet.** Per ogni frame resta un solo punto, scelto per **area** della macchia e non per vicinanza alla traiettoria. La continuità temporale è dentro il modello, non nella scelta del punto: 8 frame in ingresso, sfondo mediano, media delle heatmap su 8 finestre. Se la heatmap ha due macchie (pallina vera e pallina ferma), sopravvive solo la più grande. L'altra non arriva a nessun nostro file.
- **B, ma solo locale, in palla_locale.segui.** Ha uno stato: dagli ultimi due punti ricava velocità, previsione e cancello. Rifiuta i salti oltre il cancello. Però:
 - sceglie tra le macchie di colore, ma di TrackNet ne ha una sola;
 - vive solo nella finestra del colpo e si arrende dopo 8 frame vuoti;
 - non c'è un costo per il cambio di identità;

– l'aspetto (colore, dimensione) non entra nell'identità.

- **Gli altri passi non ricevono la traccia di** `segui`. `direzione_nascosta.py` e `velocita_rimbalzo.py` rifanno la selezione sui punti filtrati, ognuno con le sue regole: RANSAC, tracce, continuità all'indietro, primo punto più basso.

Quanta continuità temporale sfrutta la nostra pipeline (non il modello):

Dove	Come	Quanto dura
<code>segui</code>	Velocità costante dagli ultimi 2 punti; cancello $\max(0,12h; 2,5 \cdot \text{passo})$. h = altezza del giocatore in pixel	Finestra del colpo; buchi fino a 8 frame
<code>segui(esteso=True)</code>	Riaggancio entro 0,3 h per ogni frame passato dall' ultima posizione , senza previsione	Come sopra
<code>punti_dopo</code>	Una parabola (x , y nel tempo) che spiega più punti entro 6 px	0,75 s dopo la perdita
<code>tracce</code> (solo pallina in uscita)	Ultimo punto o posizione prevista; salto $\leq 0,08$ altezze dell'immagine per frame	Buchi fino a 0,5 s
<code>inizio_traiettoria</code>	All'indietro: buchi ≤ 6 frame, salto $\leq 0,3$ h/frame, stop a cambi di velocità $2,5\times$ o di direzione oltre 60°	Dal rimbalzo al colpo
<code>estendi_col_colore</code>	Macchia più vicina all'ultima posizione, raggio 12 px + 8 px per frame di buco	1,6 s dopo il colpo
<code>exit_speed</code> + <code>controlla_risultato</code>	Controllo a posteriori: il fit fisico deve spiegare i punti entro 6 px	$\sim 1/3$ s dopo il contatto
InpaintNet	Ricostruisce i buchi	Prodotto ma mai usato in <code>velocita/</code>

3. Caso di test: 5 palline ferme e 1 in movimento

Il percorso segue il codice. Dove conta il comportamento del modello TrackNet, è segnato **NON VERIFICABILE DAL CODICE**.

Prima di tutto, su tutto il video (`predict.py`). Per ogni frame TrackNet riceve 8 fotogrammi e la mediana dello sfondo. Se le 5 palline restano ferme per gran parte del video, stanno nella mediana. Se e quanto il modello le ignori dipende da quello che ha imparato: NON VERIFICABILE DAL CODICE. Sui nostri video:

- **video_alcaraz_palline_sparse** (camera alta, palline lontane): 0 punti tolti come palline ferme;
- **Giorgio** (camera a 1,71 m, palline a terra nella fascia della rete): 198 punti su 1362, cioè il 14,5%.

Quando nella heatmap ci sono più macchie sopra 0,5 vince quella di **area** maggiore.

1. **La pallina arriva verso il giocatore.** In `velocita_uscita.analizza`, finestra dalla posa ($a-10 \dots b+10$):

- `tracknet_fermi` cancella i frame in cui il punto di TrackNet è su una pallina ferma;
- `segui` parte dal primo punto di TrackNet **confermato** (un altro punto entro 2 frame e 0,3 h). Di solito è la pallina che arriva. Ma se il primo punto confermato della finestra è una pallina ferma non segnata, si parte da lì (punto 4).

2. **Ogni frame, in** `segui`:

- `previsione` = ultimo punto + velocità;
- colore: macchie giallo-verdi tonde e piene nel quadrato di lato $2 \cdot \max(60 \text{ px}; 0,3 h)$ attorno alla previsione, escluse quelle sul giallo fermo; vince la più vicina alla previsione, se entro il cancello;
- altrimenti il punto di TrackNet, se entro il cancello;
- altrimenti buco.

3. **Al contatto, la pallina è mossa o coperta:**

- il colore spesso fallisce, perché la macchia allungata dal mosso non passa i filtri di forma e riempimento della ricerca normale;
- TrackNet può indicare una pallina ferma. Se è segnata, il frame è già vuoto e resta un buco. Se **non** è segnata ed è entro il cancello, **la traccia salta sulla pallina ferma**;

- i casi tipici in cui non è segnata sono le palline grandi vicino al giocatore (punto 6). Qui conta solo la distanza dalla previsione.

4. Dopo il contatto:

- la previsione usa ancora la velocità di prima, quindi va nel verso sbagliato;
- il cancello è largo (2,5·passo), la finestra del colore invece è fissa attorno alla previsione;
- se la pallina si ritrova entro 8 frame, la traccia continua. trova_contatti cerca il cambio di velocità vicino al polso;
- misura fa il fit fisico. Se i punti sono di due palline, l'errore supera 6 px e la misura viene scartata ("misura scartata"). **Questo è l'unico vero controllo d'identità sui km/h, e arriva a posteriori.**

5. In volo la pallina passa vicino a S3.

- In velocita_uscita si guarda solo fino a $b+10$ e solo i punti fino a $1/3$ s dopo il contatto.
- Se S3 è nella maschera del giallo fermo (dilatata di 15 px), lì il colore non vede **nemmeno la pallina vera** e resta un buco.
- Se TrackNet indica S3 e S3 è segnata, buco. Se S3 non è segnata ed è entro il cancello, salto.

6. Ricomparsa: entro 8 frame la pallina è ripresa se un candidato entra nel cancello. Oltre, la traccia di segui è finita.

7. Passi successivi.

- direzione_nascosta.py (solo righe "contatto non visibile"): punti di TrackNet nei 0,75 s dopo l'ultima posizione, RANSAC. Una pallina ferma non segnata spiega bene una "parabola" costante. Se i suoi punti sono la maggioranza vince lei, poi si_allontana la scarta (la distanza dal giocatore non cresce): niente direzione sbagliata, ma il colpo resta senza direzione e si passa alla pallina in uscita.
- velocita_rimbalzo.py: il primo punto più basso, nel campo avversario, dei punti filtrati entro 1,6 s. **Nessun legame con la pallina misurata** (sezione 7).

Le domande:

- **Cosa impedisce a TrackNet di saltare sulla pallina ferma?** Nel nostro codice niente. TrackNet sceglie da solo. Possono aiutare l'ingresso con la mediana e la media delle heatmap, ma è un comportamento appreso: NON VERIFICABILE DAL CODICE. Noi interveniamo dopo: cancellando il frame (tracknet_fermi) o rifiutando il punto fuori dal cancello (seguì).
- **Cosa impedisce al rilevatore di colore di scegliere una pallina ferma?**
 - giallo_fermo: il giallo presente nella mediana di 15 fotogrammi del tratto, allargato di 15 px, viene escluso;
 - la ricerca solo attorno alla previsione;
 - i filtri su area, forma e riempimento.

Non basta per una pallina ferma solo in parte del tratto (arrivata, raccolta, calciata), che non sta nella mediana.

- **Cosa succede se la pallina vera si perde per k frame:**

k	seguì normale	seguì estesa	Altri passi
1-3	Previsione a velocità costante, cancello che cresce con k: di solito riacquisita	Riaggancio a TrackNet entro $0,3 \cdot k$ h dall'ultima posizione ($0,9$ h con $k = 3$)	—
5	Come sopra, ma la finestra del colore resta fissa: dopo un cambio di direzione (contatto, rimbalzo) guarda nel posto sbagliato	Riaggancio entro 1,5 h : può prendere un'altra pallina in movimento o una ferma non segnata	inizio_traiettoria accetta buchi fino a 6
8	Ultimo frame tollerato ($BUC0_MAX = 8$; $0,27$ s a 30 fps, $0,13$ s a 60 fps)	Riaggancio entro 2,4 h	—
10	La traccia finisce al 9° frame vuoto e non riparte	Finisce anche lei (stesso $BUC0_MAX$)	punti_dopo cerca ancora per 0,75 s; tracce tollera 0,5 s; il colore lontano allarga il raggio di 8 px per frame

- **Quando viene riacquisita, come si decide che è la stessa pallina?** Solo dalla geometria:

- vicinanza alla previsione (seguì);
- vicinanza all'ultima posizione (estesa, colore lontano);
- curva liscia (RANSAC; rimbalzo ricostruito con le due curve entro 20 px);
- continuità (inizio_traiettoria, tracce);
- fit fisico a posteriori.

Nessun confronto di aspetto, nessuna ipotesi alternativa.

- **Esiste una distanza massima dalla traiettoria precedente?** Sì, ma diversa in ogni passo (tabella della sezione 2).
- **Viene considerata la velocità o direzione precedente?** Sì in `seguì` (velocità costante), tracce (posizione prevista) e `inizio_traiettoria` (cambio 2,5x, 60°). No nel riaggancio esteso e nel colore lontano.
- **Viene considerata la posizione del giocatore o della racchetta?**
 - Giocatore sì: finestre dalla posa, altezza per le soglie, polso per il contatto (0,6 h) e per la pallina in uscita (0,8 h), piedi in `si_allontana`, riquadro in `seme_lancio`.
 - Racchetta **no**.
- **Viene considerato il contatto?** Sì come `ancora`: misura usa i punti dopo il contatto; `velocita_rimbalzo` non risale prima della riga se il contatto è stato trovato. Però il punto del rimbalzo del passo 6 non è legato ai punti misurati.

4. Dopo ogni colpo: c'è uno stato?

No. Ogni colpo riparte da zero. In `velocita_uscita.analizza`, il ciclo `for a, b in finestre_colpi(...)` fa per ogni finestra, da capo:

1. `tracknet_fermi(primo, ultimo)`;
2. il filtro dei punti (`tn`);
3. `seguì(primo, ultimo, tn)` **senza seme**;
4. `trova_contatti`.

Da una finestra all'altra passano solo ferme (accumulato per il file) e controllati. La pallina del colpo prima non viene passata al colpo dopo.

- **Identificazione al colpo 2:** il seme è il primo punto confermato da `a-10`. La "pallina effettivamente colpita" è l'oggetto della traccia che cambia verso vicino al polso. Al momento del colpo non c'è una scelta tra più palline.
- **Fine della traiettoria:** la traccia finisce a `b+10` o dopo 8 frame vuoti. Il rimbalzo o il colpo dell'avversario non chiudono niente in `velocita_uscita`. `velocita_rimbalzo` guarda fino a 1,6 s dopo la riga, con metodi suoi.
- **Stato tra colpi:**
 - usati in `velocita_rimbalzo` (un rimbalzo non vale per due colpi);
 - le regole contro le righe doppie (`DISTANZA_MINIMA` 20 frame, 15 frame, `STESSA_FINESTRA`).
- **Senza finestra della posa la pallina non si cerca:** in `velocita/` un colpo esiste solo se lo vede la posa.

Rischio dedotto dal codice, non provato su video. A ritmo di cesto (circa 2-3 s tra i colpi nei nostri video) nella finestra del colpo 2 possono esserci sia la pallina del colpo 1 sia la nuova. Se il primo punto confermato è la pallina vecchia, `seguì` la segue finché resta nel cancello e poi la perde. Senza la pallina vicino al polso, `pallina_vicina` non arriva a 3 punti: **il colpo può non avere nemmeno la riga**.

5. Una, tre, cinque, dieci palline ferme

Il codice non ha nessuna logica che dipenda dal numero di palline ferme. Non esiste un elenco delle palline ferme: ogni frame si giudica da solo.

Palline ferme	Cosa cambia
1	Pochi frame in cui TrackNet può sceglierla. Il filtro la toglie se è piccola, in zona e ferma 0,4-0,6 s prima e dopo.
3	Più frame su palline ferme. La maschera del giallo fermo copre più superficie: più punti in cui il colore non vede la pallina vera che passa vicino.
5	Come sopra. Cresce la probabilità che il seme o un riaggancio esteso cadano su una pallina ferma non segnata (vicina alla camera, appena arrivata, fuori zona).
10	Con palline vicine tra loro (gruppi, cesto): macchie fuse (area oltre 120 px ² , non contate) e punti di TrackNet in mezzo al gruppo. Aumenta anche il rischio, raro, di togliere la pallina vera (sezione 6).

Distinzione progettata o effetto collaterale? In parte progettata: `tracknet_fermi` e `giallo_fermo` sono nati apposta (3 ottobre, LEGGIMI "Palline ferme in campo"). Ma sono **filtri sulle detection**: la "pallina in gioco" non è un oggetto del programma, è quello che segue `seguì`. Altre protezioni ci sono per effetto collaterale: il cancello, la RANSAC, `si_allontana`, l'errore massimo del fit.

Dati dei 7 video (catena attuale; punti di TrackNet = frame con fonte "tracknet"):

Video	Frame	Con punto TrackNet	Tolti come palline ferme
Giorgio	1897	1362 (72%)	198 (14,5%)
video_alcaraz_palline_sparse	1902	1210 (64%)	0
alcaraz	1315	850 (65%)	0
swing_vision	1507	625 (41%)	0
test_tennis_1	480	324 (68%)	1
Nicola	320	192 (60%)	0
Djokovic	263	205 (78%)	0

6. Il filtro delle palline ferme

Cosa verifica esattamente (`palla_locale.tracknet_fermi`):

- **Punti controllati:** solo quelli di TrackNet che, portati a terra (`a_terra_in_campo`), cadono in x da -7 a 18 m e y da -8 a 34 m. I pixel sopra l'orizzonte non si controllano.
- **Per ogni punto P al frame f**, e per $d = 0,4$ s e $0,6$ s:
 - **test largo:** nei frame $f-d$ e $f+d$ si cercano macchie gialle piccole (HSV 22-48, area 3-120 px² a 1080p, scalata con la risoluzione) entro un raggio $h \cdot (0,02 + 0,10 \cdot d)$ da P. Fa 0,06 h a 0,4 s e 0,08 h a 0,6 s; h è l'altezza del riquadro del giocatore;
 - **test stretto:** serve una macchia A prima e una B dopo con il punto medio $(A+B)/2$ entro 0,02 h da P.
- **Se uno dei due d passa**, si cancella **tutto il frame** (cioè l'unico punto di TrackNet).
- **Dove:** finestre dei colpi ± 10 frame, più 1,6 s dopo ogni riga (`FERME_DOPPO_S`). I frame vanno in `<video>_palline_ferme.csv`, che `direzione_nascosta.py` e `velocita_rimbardo.py` leggono per frame.

Falsi positivi che elimina: palline a terra ferme o che rotolano piano e dritte ($\leq 0,1$ h/s), piccole (lontane), nella zona del campo, visibili 0,4-0,6 s prima e dopo. Su Giorgio 198 punti. Controllo a occhio del 3 ottobre (`COME_FUNZIONA 8.10`): 16 punti tolti su 18 erano palline a terra nella fascia della rete.

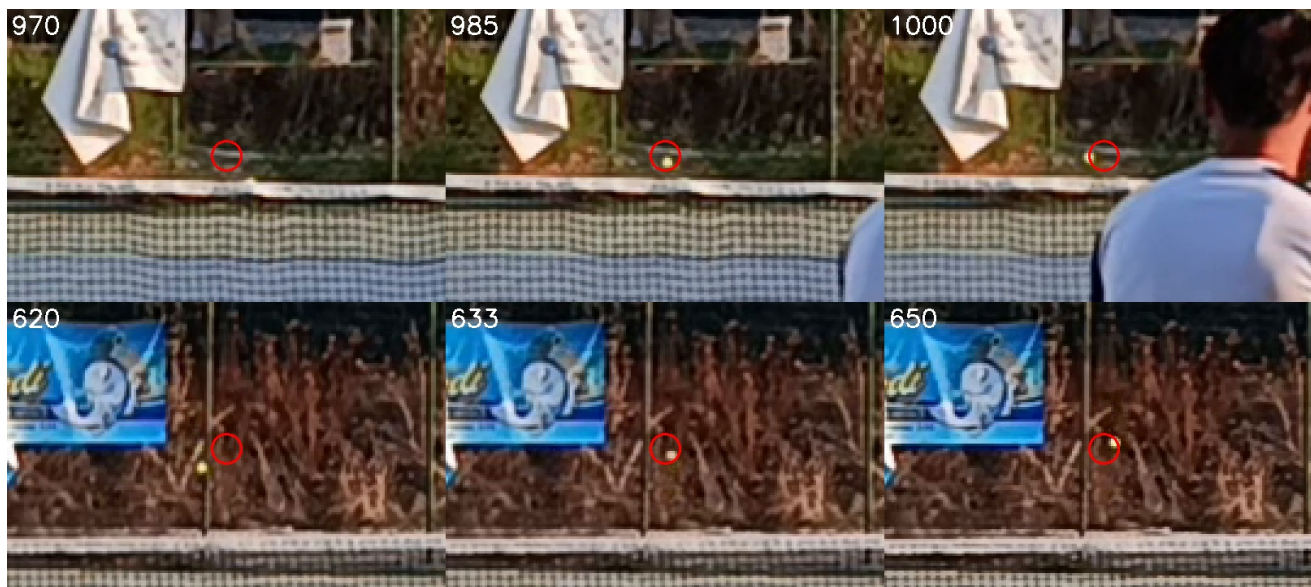
Casi che NON elimina:

1. **Palline grandi, cioè vicine alla camera:** area oltre 120 px² a 1080p. Con un giocatore alto 500 px una pallina alla sua distanza ha un diametro di circa 19 px, quindi un disco di circa 270 px²; la soglia di colore prende spesso solo il centro, quindi la macchia può essere più piccola. **Le palline ferme nella metà campo del giocatore, dove avviene il contatto, rischiano di non essere**

segnate (dal codice; NON VERIFICATO su video). Per il colore sono comunque escluse da giallo_fermo, se ferme nella mediana.

2. **Palline ferme da meno di 0,4-0,6 s, raccolte o calciate:** manca la macchia prima o dopo.
3. **Palline che rotolano più veloci di 0,1 h/s o che curvano.**
4. **Palline fuori zona.** Con la camera bassa i punti vicino alla recinzione di fondo finiscono oltre 34 m e non vengono controllati. Verificato su Giorgio:
 - frame 983-987: punto proiettato a circa 43 m; togliendo il limite della zona il test li segnerebbe;
 - frame 632-635: circa 85 m.

La zona era nata per non togliere la pallina in volo davanti ai cespugli gialli.



Giorgio, punti di TrackNet su palline vicino alla recinzione di fondo che il filtro non controlla (cerchio rosso = punto di TrackNet). Sopra: frame 970, 985, 1000 (pallina arrivata alla recinzione: al 970 non c'è ancora). Sotto: frame 620, 633, 650 (pallina che si muove piano).

5. **Pallina coperta** (giocatore davanti) a f–d o f+d.
6. **Colore fuori dalla soglia HSV** (luce, palline vecchie).
7. **Frame fuori dagli intervalli controllati:**
 - velocita_rimbalzo.stima comincia la ricerca a fc–25 frame, che può cadere prima della finestra;
 - le righe spostate dalla pallina in uscita: su Giorgio 14,0 s il filtro arriva al frame 453, la ricerca del rimbalzo al 470.

Può eliminare per errore la pallina vera? Sì, in tre casi:

- **(a) La pallina in gioco passa, nell'immagine, sopra una pallina ferma.** Il test passa e quel frame si cancella: resta un buco di 1-2 frame, non un salto.
- **(b) Due palline ferme quasi simmetriche rispetto al punto:** entro 0,06-0,08 h, punto medio entro 0,02 h. Raro, più probabile nei gruppi.
- **(c) Una pallina in gioco lenta e dritta nell'immagine per $\pm 0,4-0,6$ s.** Con la versione attuale NON VERIFICATO sui video.

2-3 palline ferme vicine. Se TrackNet indica una di loro, la macchia prima e dopo è la stessa e il punto medio coincide: il filtro funziona. Se le palline si toccano nell'immagine, la macchia è unica: il centro cade in mezzo e l'area può superare 120 px². In quel caso non viene segnata.

Vale anche per velocita_rimbalzo.py? Sì, per frame: `tn = {f: p ... if f not in ferme} in aggiorna.` Ma i passi con il colore (`estendi_col_colore`, `macchie_lontane`) **riempiono proprio i frame tolti**. Usano la mediana del tratto e nessun filtro di forma, quindi possono rimettere la stessa pallina ferma, se non sta nella mediana.

Percorsi che saltano il filtro:

1. `classificazione/rileva_impatti.py` e `metriche_colpi.py`: punti grezzi.

2. `analyze.ball_speeds` (`pallina_velocita_px_s`): punti grezzi.
3. I rilevatori di colore (`palla_locale.candidati`, `velocita_rimbalzo.macchie_lontane`) usano `giallo_fermo`, un'altra definizione di "fermo".
4. Risultati vecchi senza `<video>_palline_ferme.csv`: `leggi_palline_ferme` avvisa e usa tutti i punti.
5. I frame fuori intervallo del punto 7 sopra.

7. Incoerenze tra le parti

#	Incoerenza	Effetto possibile
1	Due definizioni di "pallina ferma": <code>tracknet_fermi</code> (punti di TrackNet, macchie piccole, zona del campo) e <code>giallo_fermo</code> (colore, mediana del tratto, qualsiasi dimensione).	Una pallina ferma grande vicino al giocatore è esclusa per il colore, ma i punti di TrackNet su di lei restano validi per <code>seguì</code> , <code>direzione_nascosta</code> e <code>velocita_rimbalzo</code> .
2	Ogni file rifà la selezione. <code>velocita_rimbalzo</code> non riceve né la traccia né i <code>_punti</code> del colpo.	Il rimbalzo di un colpo può essere scelto su punti di un'altra pallina.
3	Passo 6 di <code>velocita_rimbalzo</code> : il primo punto più basso nel campo avversario entro 1,6 s, senza continuità con i punti misurati.	Può essere il rimbalzo di un'altra pallina: cesto, avversario, colpo prima. Con PUNT04 quel rimbalzo dà anche i km/h dei colpi "solo direzione" e la velocità media (lì c'è il controllo <code>media ≤ uscita</code>).
4	Riaggancio con soglie diverse: estesa 0,3 h/frame dall'ultima posizione; colore lontano 12 + 8 px/frame; RANSAC 6 px; tracce 0,08 altezze dell'immagine/frame.	La stessa pallina può essere "ripresa" in un passo e "persa" in un altro.
5	Un frame tolto come pallina ferma può tornare con il colore in <code>velocita_rimbalzo</code> (estendi_colore riempie i frame senza punto).	Una parte scarta la pallina, un'altra può riprenderla.
6	Pipeline parallela della classificazione (<code>rileva_impatti</code> , <code>metriche_colpi</code>) su punti grezzi e interpolati.	Due elenchi di colpi e due metriche diverse dallo stesso video.
7	Intervallo del filtro diversi da quelli di ricerca (<code>fc-25</code> ; righe spostate).	Pochi frame non filtrati usati nella ricerca del rimbalzo.
8	I punti "stimati" di InpaintNet: ignorati sia in <code>velocita/</code> sia in <code>rileva_impatti</code> (che però interpola da sé).	Coerente. È tempo di calcolo che nessuno usa.

8. Detection o tracking?

- **Detection (vedo una pallina qui):**
 - TrackNet, un punto per frame;
 - `palla_locale.candidati`, `macchie_gialle`;
 - `velocita_rimbalzo.macchie_lontane`.
- **Tracking (è la stessa di prima):**
 - `palla_locale.seguì`: un'ipotesi sola, il candidato più vicino alla previsione entro un cancello, velocità costante, solo nella finestra del colpo, al massimo 8 frame di buco;
 - `direzione_nascosta.tracce`: solo per la pallina in uscita;
 - `velocita_rimbalzo.inizio_traiettoria`: all'indietro;
 - `estendi_colore`: il più vicino, senza previsione.
- **Controlli di traiettoria**, non tracking ma coerenza a posteriori: RANSAC, `exit_speed` + `controlla_risultato` (6 px), `direzione_in_uscita` (4 px, 6°), rimbalzo ricostruito (20 px).
- **Manca:**
 - un'identità della pallina che duri per tutto il video;
 - una scelta tra più candidati di TrackNet nello stesso frame;
 - uno stato tra un colpo e il successivo;
 - "pallina in gioco" e "palline ferme" come oggetti distinti.

In sintesi: abbiamo detection più un tracking locale a ipotesi singola, a pezzi, ricostruito da capo in ogni file.

9. Test pratici

Per tutti i test:

- Telefono nella posizione standard, o calibrazione del video; meglio 60 fps; 30-60 s per video.
- MOSTRA_TRACKING = True in disegna_velocita.py.
- Una **lista dei colpi veri fatta a occhio**: frame del contatto, tipo, dove rimbalza.

Cosa guardare:

- **Nel video** _velocita.mp4: anelli gialli = punti di TrackNet; rossi = punti usati per il calcolo; grigi = punti tolti come palline ferme.
- **Nei dati:** _velocita.csv (righe, nota, errore_px, rimbalzo_trovato_frame), _velocita_punti.json (_punti di ogni colpo), _palline_ferme.csv, _tracking.csv.

Regola d'oro per tutti: un solo punto rosso su una pallina ferma = FAIL di identità, anche se i km/h sembrano giusti.

Test	Come girarlo	Cosa osservare	PASS	FAIL
A 1 in gioco, 0 ferme	Campo sgombro, 10-15 colpi a cesto, palline raccolte subito	Riferimento: righe, errore_px, punti rossi	Ogni colpo vero ha la sua riga; i punti rossi sono tutti sulla pallina colpita; _palline_ferme.csv vuoto o quasi; errore ≤ 3 px a 1080p sui colpi misurati	Punti rossi fuori dalla pallina; colpi veri senza riga anche qui (il problema non sono le palline ferme)
B 1 in gioco, 3 ferme	Come A, con 3 palline ferme messe apposta: una nella metà campo del giocatore a 1-2 m dal punto d'impatto, una a metà del volo, una vicino a dove rimbalza	Anelli grigi; punti rossi vicino alle 3 palline; rimbalzo_trovato_frame: in quel frame la pallina è quella colpita?	Nessun punto rosso sulle ferme; anelli grigi solo sulle ferme (mai sulla pallina in gioco); stesse righe e km/h di A entro $\pm 10\%$ per colpi simili; rimbalzi sulla pallina giusta	Punti rossi sulla ferma vicina al giocatore (non viene segnata perché grande); rimbalzo trovato su una ferma; più "misura scartata" che in A
C 1 in gioco, 5-10 ferme	5-10 palline sparse, più un gruppo di 2-3 vicine e una vicino alla recinzione di fondo	Come B, più: i punti gialli fermi per più di 0,5 s senza anello grigio sono palline ferme non tolte	Come B; nessuna riga in più o in meno rispetto ai colpi veri	Righe false o perse; punti rossi sul gruppo o sulla pallina alla recinzione
D Passaggio molto vicino a una ferma	Una pallina ferma sulla traiettoria del colpo (metà campo avversario) e una dove passa la pallina in arrivo	I frame in cui la pallina in gioco passa entro ~ 20 px dalla ferma	I punti rossi continuano sulla pallina in gioco; al massimo 1-2 frame vuoti; nessun anello grigio sulla pallina in gioco per più di 2 frame	La traccia resta sulla ferma; la direzione cambia rispetto al colpo simile senza la ferma
E Occlusione o mosso al contatto	Roveschi con la pallina dietro il corpo, colpi veloci con poca luce, una pallina ferma ai piedi del giocatore	Righe "contatto non visibile", "solo direzione", "pallina in uscita"; da dove vengono i punti della direzione	Ogni colpo ha la riga; se c'è una direzione, i punti usati sono della pallina colpita; nessuna direzione da una pallina ferma	Direzione o rimbalzo presi dalla pallina ai piedi; riga senza dati quando la pallina si vede bene
F (in più) Colpi consecutivi	Cesto a 2-2,5 s tra i colpi, 15-20 colpi	_punti del colpo n; rimbalzo_trovato_frame del colpo n	Ogni colpo ha la sua riga; _punti del colpo n non contengono la pallina del colpo n-1 o quella lanciata dopo; rimbalzo del colpo n prima del contatto del colpo n+1	Colpi senza riga (seme sulla pallina vecchia); rimbalzo del colpo n preso dal colpo n+1

Per dichiarare PASS su un test servono almeno 10 colpi e nessun FAIL di identità (regola d'oro). Gli altri criteri si contano in percentuale e si confrontano con il test A.

10. Verdetto

REQUISITO: "Ad ogni colpo identificare e seguire la pallina effettivamente colpita anche in presenza di molte palline ferme."

VERDETTO: PARZIALMENTE IMPLEMENTATO

Cosa abbiamo già:

- TrackNetV3 con sfondo mediano e media temporale delle heatmap.
- Un tracker locale nella finestra di ogni colpo (seguì): previsione, cancello, colore dove TrackNet perde la pallina, giallo fermo escluso.
- Un filtro esplicito delle palline ferme sui punti di TrackNet (`tracknet_fermi`), applicato in modo coerente, per frame, in tutti e tre i file di velocità/.
- Contatto legato al polso e controllo fisico a posteriori (6 px), che scarta le misure fatte su punti di palline diverse.
- Controlli di traiettoria nei passi successivi: RANSAC, `si_allontana`, tracce, rimbalzo ricostruito.

Cosa manca:

- Più candidati per frame da TrackNet: oggi la pallina vera, se TrackNet sceglie la ferma, in quel frame non esiste.
- Un'identità della pallina che attraversi i passi: contatto → volo → rimbalzo, nello stesso file e da un file all'altro.
- Uno stato tra un colpo e l'altro, e una scelta esplicita della pallina colpita quando più palline sono vicine al giocatore.
- Il filtro sulle palline ferme vicine alla camera (grandi) e su quelle fuori zona con la camera bassa.
- Il legame tra il rimbalzo del passo 6 (e PUNT04) e la traiettoria misurata.

Principale punto debole. Il punto unico per frame di TrackNet, scelto per area, prima di qualunque tracking. Tutto il resto del sistema può solo cancellare o rifiutare quel punto, mai sostituirlo con la pallina giusta.

Modifica minima necessaria (solo proposta, non implementata).

1. In `tracknet3/predict.py`, in fase di decodifica e senza riaddestrare: salvare in un file a parte **tutte le macchie** sopra soglia di ogni frame (centro, area, picco), non solo la più grande.
2. In `palla_locale.seguì`: usarle come candidati, accanto al colore. Ogni frame si sceglie il più vicino alla previsione entro il cancello. I candidati che `tracknet_fermi` riconosce come fermi si scartano uno per uno, non più per frame.

Pro: affronta il punto debole senza cambiare il resto della catena. Contro: va rifatto TrackNet su tutti i video (circa 20 minuti per un 4K su CPU) e va verificato con l'A/B sui 7 video.

Un passo ancora più piccolo, che da solo **non basta** per il requisito ma toglie l'incoerenza 3: in `velocita_rimbalzo`, cercare il rimbalzo del passo 6 (e di PUNT04) solo sulla traccia (`tracce`) che contiene i punti del colpo.